

STUDYSYNC SA

PART 2 – APP PROTOTYPE DEVELOPMENT

Submission Report

Student name: Devon Adams
Student number: ST10439194
Module: OPSC6312
Institution: Rosebank College

1. Introduction and purpose

StudySync SA is an Android academic productivity application designed to help students organise academic responsibilities in one place. It brings together modules, tasks, notes, deadlines, reminders and progress-related information so students can keep track of work and plan study time.

This report documents the prototype developed for Part 2, including its purpose, design, functionality, technical implementation, API and database integration, testing, and use of GitHub and GitHub Actions. Replace all bracketed prompts and insert screenshots before submission.

2. Application overview and requirements

StudySync SA was designed for students managing several modules and overlapping deadlines. The application aims to provide a clear place to capture academic tasks and notes, review upcoming work and customise preferences.

2.1 Intended users

- Primary users: students who need to organise module-related academic work.
- Typical use: create and review tasks, maintain notes, check due dates and adjust preferences.

2.2 Part 2 requirements checklist

- App builds and runs on a supported Android device/emulator.
- Registration and login work; password handling is secure.
- Users can change settings.
- App connects to a hosted REST API and database.
- Implemented Part 1 features are demonstrated.
- Invalid input is handled without crashing.
- Automated tests and GitHub Actions build/test workflow are included.
- Source code and README are in GitHub.

3. Design considerations and user interface

The interface should make common student actions easy to locate and complete. Screens should use clear labels, consistent styling, readable text and touch-friendly controls. Provide useful feedback after actions and validation messages that explain how to correct invalid input.

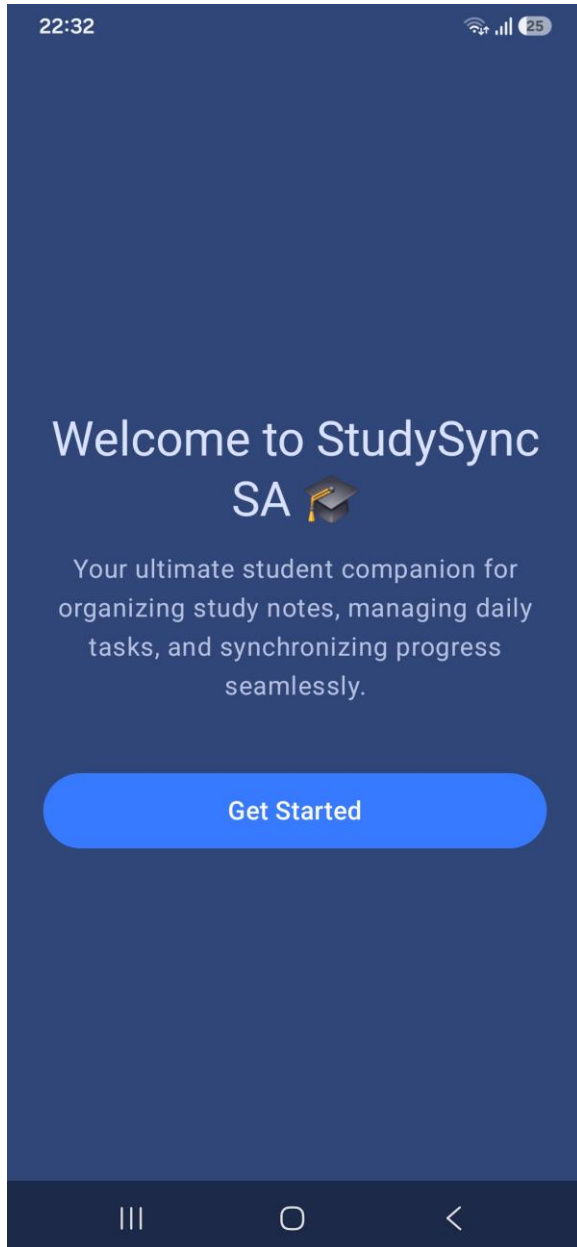


Figure 1: App launch/ Welcome Screen

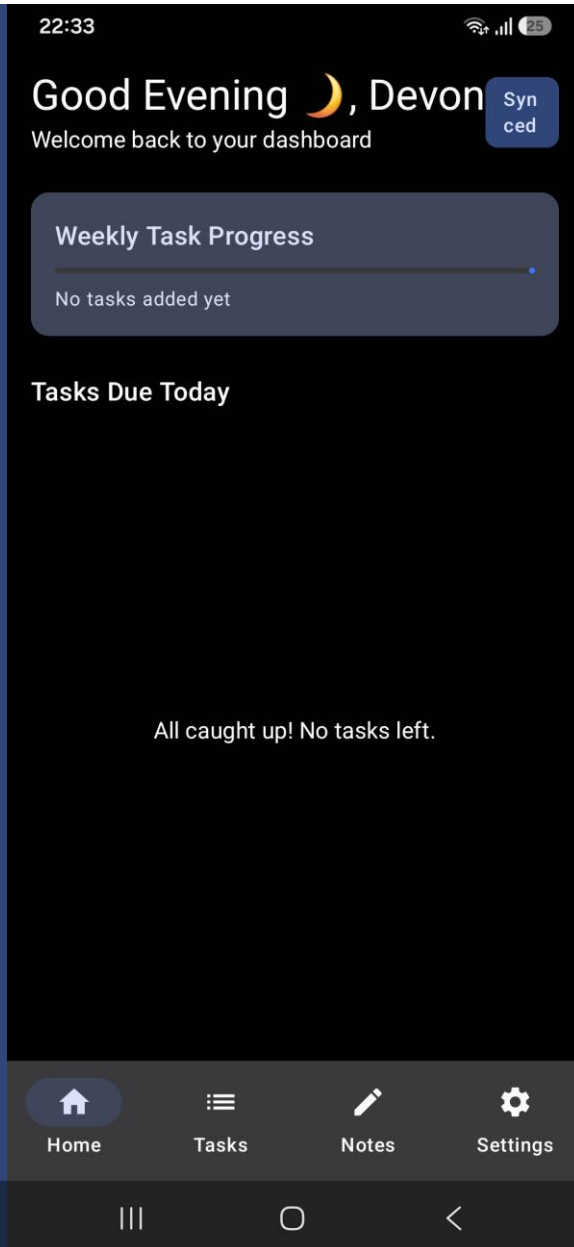


Figure 2: Dashboard/ Main Navigation

3.1 Usability considerations

- Clear navigation between main areas.
- Consistent labels and visual styling.
- Readable text, suitable spacing and touch-friendly controls.
- Feedback for saving, updating and deleting records.
- Validation and error messages that are understandable.

4. Main functionality

Describe and demonstrate only features present in the final prototype. Adjust the descriptions below if your implementation differs.

4.1 Registration and login

The authentication flow allows users to create an account and sign in. Credentials should be managed by the selected authentication provider; do not store or display plaintext passwords.

Figure 3: registration Screen

Figure 4: Login Screen

4.2 Tasks, priorities and deadlines

Explain implemented task actions, such as creating or editing tasks, associating them with modules, setting priorities or due dates, and tracking completion.

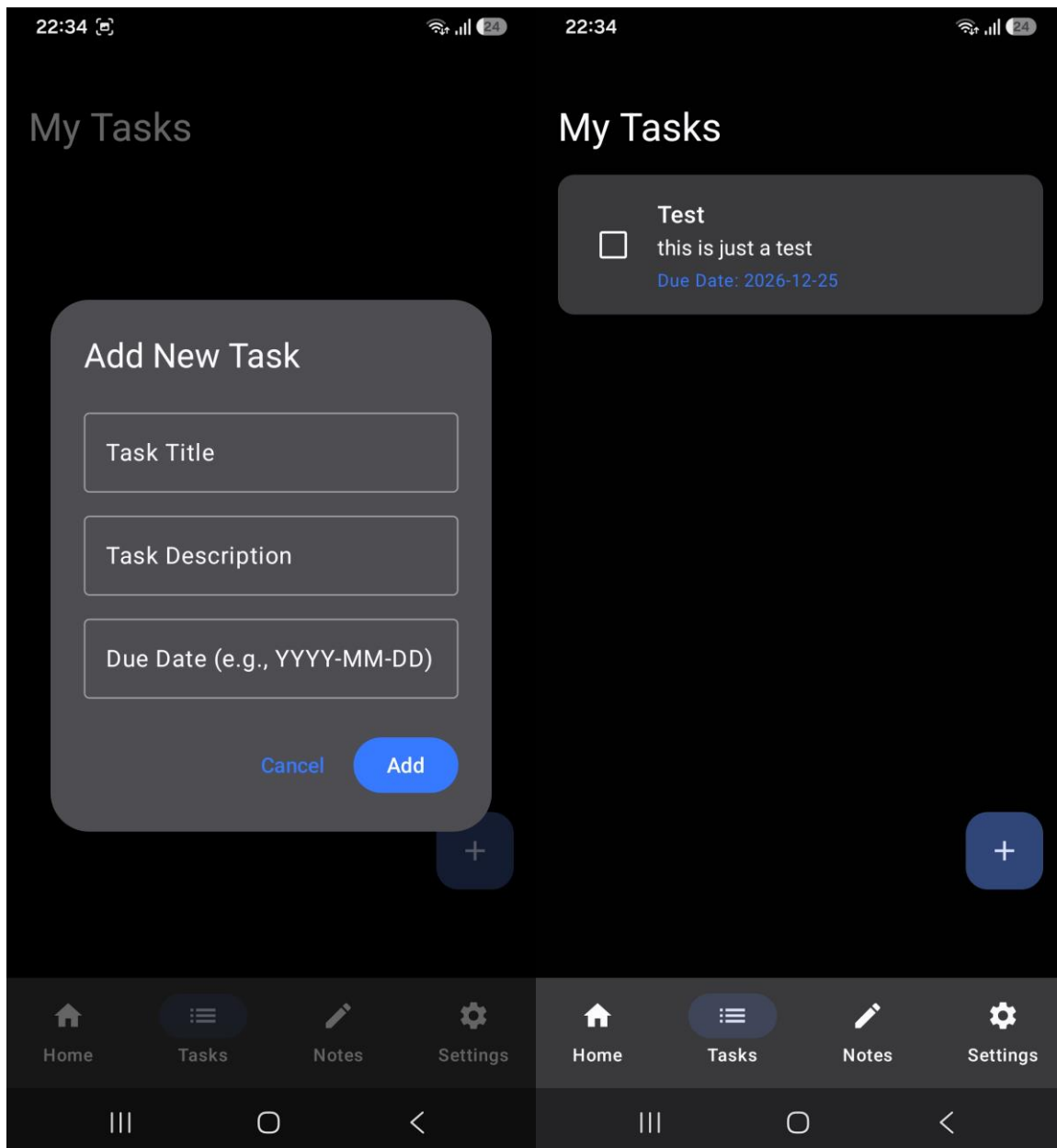


Figure 5: Add or Edit Task form

Figure 6: Task List/ Task Details

4.3 Notes and search

Describe how users create, view and organise notes. Mention search/filtering only if implemented.

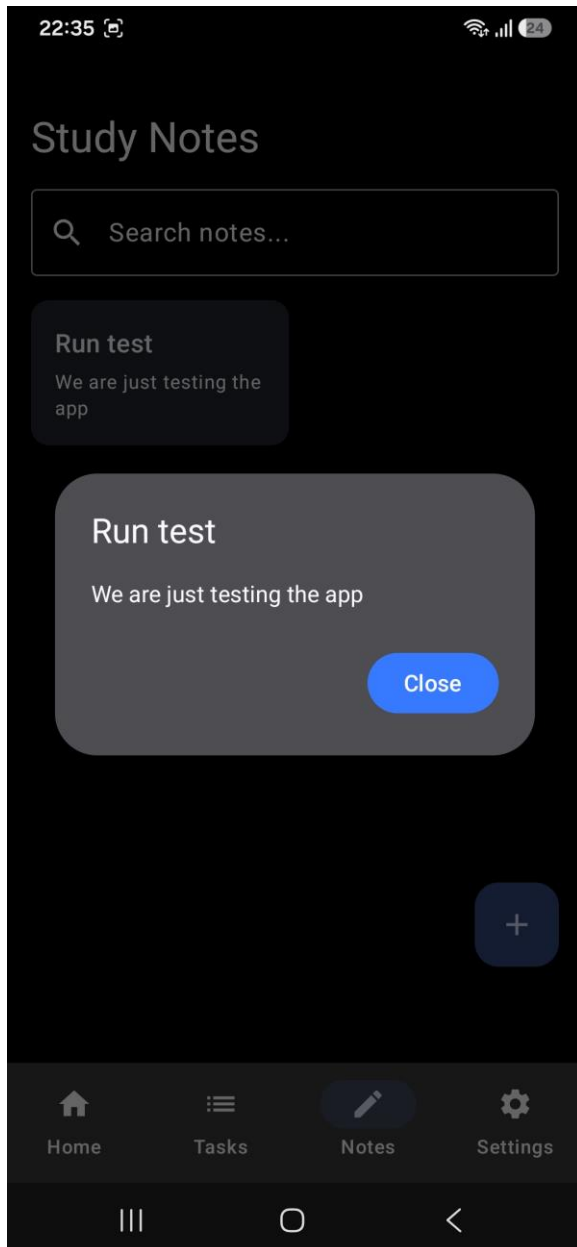


Figure 7: Notes list/ Opened Note

4.4 Reminders and progress

Describe any reminder, weekly goal, streak, progress or quick-add functionality that is implemented. Identify unfinished functions as future work.

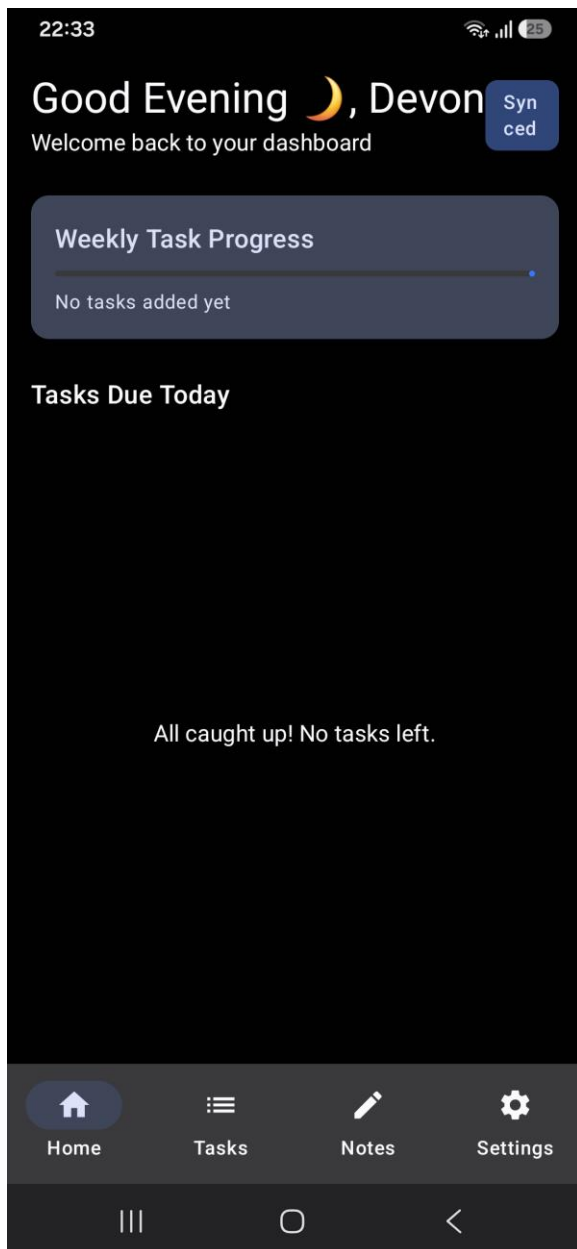


Figure 8: Reminder/ Progress Screen

4.5 Settings and preferences

Describe implemented settings, such as language, theme, notifications or study goals. List only preferences that have been tested.

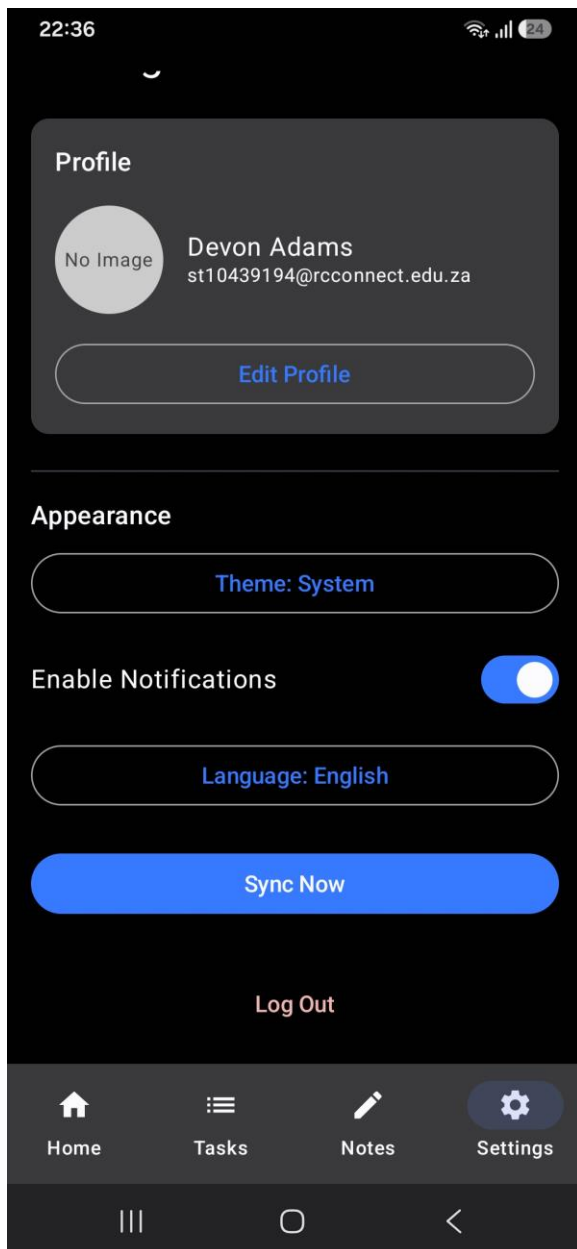


Figure 9: Settings Screen

5. Technical architecture and implementation

The design specifies an Android client using Kotlin and Jetpack Compose. Local persistence may use Room/SQLite. The backend design uses a REST API connected to hosted PostgreSQL. Confirm the technologies below against the actual application.

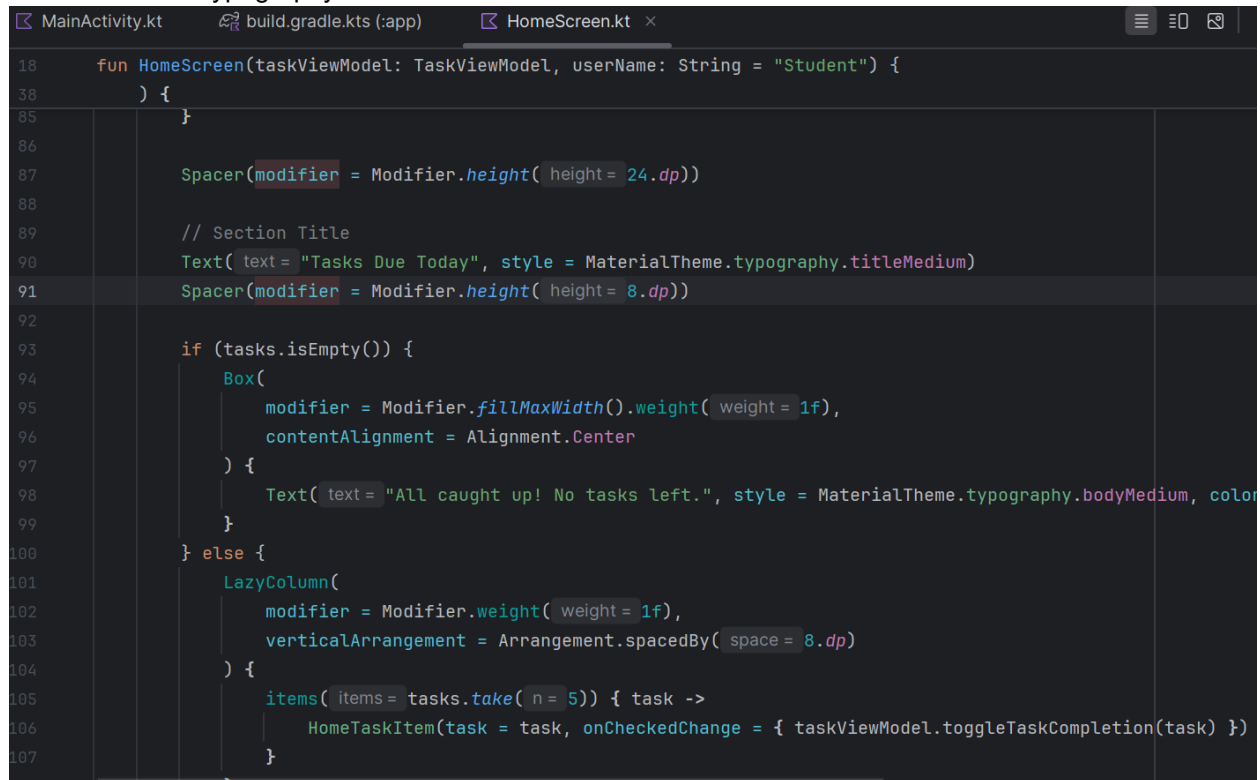
5.1 Code structure

[Describe the actual packages, screens, data models, repositories/services and network flow. Include important class names from your project.]

1. **Package Structure Overview** The project follows clean architecture principles tailored for an offline-first Android application using modern Jetpack Compose. The packages are organized logically as follows:
 - The project follows clean architecture principles tailored for an offline-first Android application using modern Jetpack Compose. The packages are organized logically as follows:
 - `com.studysync.sa`
 - `MainActivity.kt`: The application entry point that manages the top-level app state, user theme settings, the welcome screen state, and core user authentication state.
 - `data/` (Data Layer)
 - `local/`: Contains the local Room database (`StudySyncDatabase.kt`), entity definitions (`TaskEntity.kt`, `NoteEntity.kt`), and data access objects (`StudySyncDao.kt`).
 - `remote/`: Declares network interfaces (`StudySyncApiService.kt`) and serialization data transfer objects for syncing updates to cloud REST endpoints.
 - `repository/`: Holds `StudySyncRepository.kt`, the single source of truth managing offline-first caching logic and triggering sync workers.
 - `ui/` (Presentation Layer)
 - `navigation/`: Manages application routes and configuration parameters (`Screen.kt`).
 - `screens/`: Holds modular Jetpack Compose views for discrete application modules (`WelcomeScreen`, `AuthScreen`, `HomeScreen`, `TasksScreen`, `NotesScreen`, `SettingsScreen`).
 - `theme/`: Configures app styling, adaptive color palettes, Material 3 configurations, and fonts (`Theme.kt`).
 - `viewmodel/` (State Management Layer)
 - Architectural view models (`TaskViewModel.kt`, `NoteViewModel.kt`) that expose state streams cleanly to views via Kotlin Flow collectives.
 - `worker/` (Background Processing)
 - `SyncWorker.kt`: Leverages Android WorkManager to reliably synchronize queued local database adjustments with remote APIs in the background.
2. **Architectural Workflow Patterns** **Offline-First Strategy** When a user adds notes or modifies tasks, the operation is handled locally first for immediate responsiveness:
 - The ViewModel interacts with `StudySyncRepository`.
 - The repository updates the local Room database (`StudySyncDatabase`).
 - Simultaneously, a pending synchronization log (`SyncOperationEntity`) is inserted locally, and `SyncWorker` is queued via `WorkManager`.
 - The background worker attempts to transmit updates to the remote REST interface (`StudySyncApiService`). On success, it updates the operation status to `SYNCED`.

Authentication & Session Gate

- Upon application launch, MainActivity reads the state flag showWelcomeScreen to show an introductory interface.
- Once dismissed, the user interacts with AuthScreen, which uses Firebase Authentication to register or validate credentials before exposing application modules.
- Dynamic Material 3 UI Theme System
- User theme preferences ("System", "Light", "Dark") are managed centrally via state parameters inside MainActivity.kt.
- These are propagated to components across screens through StudySyncSATheme, ensuring that structural layouts, input text fields, and list components automatically adapt their colors and typography.



```
18 fun HomeScreen(taskViewModel: TaskViewModel, userName: String = "Student") {
38 } {
85 }
86
87 Spacer(modifier = Modifier.height( height = 24.dp))
88
89 // Section Title
90 Text( text = "Tasks Due Today", style = MaterialTheme.typography.titleMedium)
91 Spacer(modifier = Modifier.height( height = 8.dp))
92
93 if (tasks.isEmpty()) {
94     Box(
95         modifier = Modifier.fillMaxWidth().weight( weight = 1f),
96         contentAlignment = Alignment.Center
97     ) {
98         Text( text = "All caught up! No tasks left.", style = MaterialTheme.typography.bodyMedium, color = Color.Gray)
99     }
100 } else {
101     LazyColumn(
102         modifier = Modifier.weight( weight = 1f),
103         verticalArrangement = Arrangement.spacedBy( space = 8.dp)
104     ) {
105         items( items = tasks.take( n = 5)) { task ->
106             HomeTaskItem(task = task, onCheckedChange = { taskViewModel.toggleTaskCompletion(task) })
107         }
108     }
109 }
```

Figure 11: Android Studio project/package structure

6. REST API and database integration

Describe how the Android client communicates with the hosted API, how requests are authenticated, and how responses are presented. Provide only safe evidence; never include secret keys or connection strings.

The application implements an offline-first architecture by using a local Room SQLite database as the definitive single source of truth. All user modifications—such as adding tasks or editing study notes—are written immediately to local database tables, and the presentation layer remains completely functional and responsive without active network access by binding Jetpack Compose layouts directly to reactive data streams (Flow). Concurrently, every change is appended to a local write-ahead synchronization log (SyncOperationEntity) as a serialized JSON payload string marked with a "PENDING" operational state.

To synchronize data with the cloud, Android WorkManager activates a background SyncWorker as soon as the device satisfies network connectivity constraints. This background worker retrieves the sequential queue of pending operations from the local database and makes network requests via Retrofit to your live web service endpoint (<https://jsonplaceholder.typicode.com/posts> for safe mock testing). When the REST API responds with a successful HTTP status code, the operation is updated locally to a status of "SYNCED", guaranteeing eventual consistency between the client and your database storage layers.

7. Version control and continuous integration

The project should be maintained in GitHub with a README describing the app and linking to the demonstration. GitHub Actions should run configured build and test steps. Complete these details using your actual repository.

Repository URL: [blackman-21/StudySyncSA \(github.com\)](https://github.com/blackman-21/StudySyncSA)

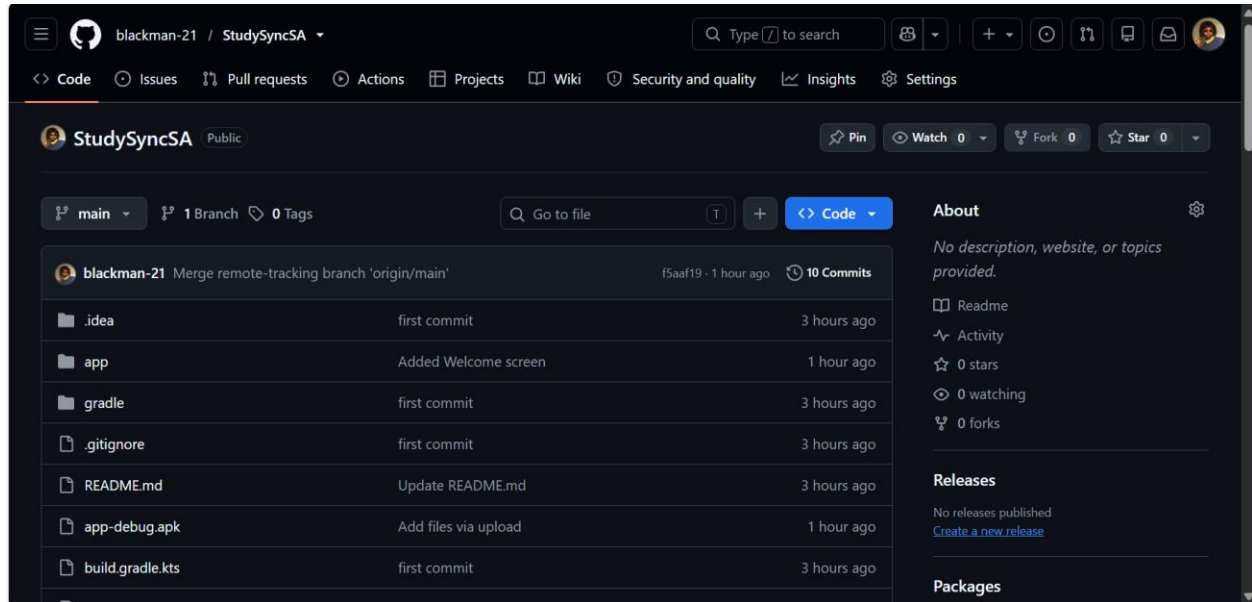


Figure 12: GitHub Repository

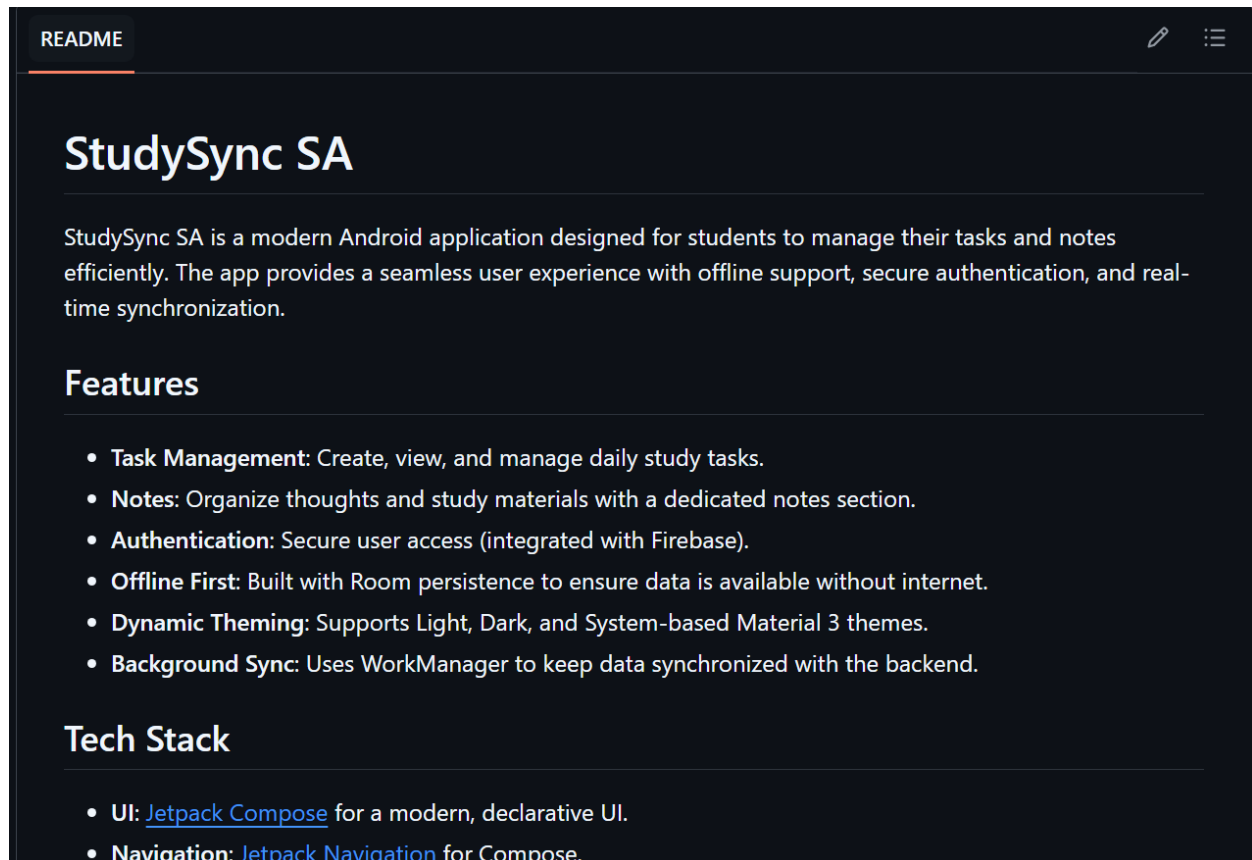


Figure 13: README File

8. Conclusion

StudySync SA is designed to help students manage academic tasks and information through an Android application connected to backend services. This report records the prototype's implementation and provides locations for evidence of its interface, persistence, testing and development workflow. Before submission, replace every bracketed prompt, insert screenshots, verify claims against the final build, and check that the repository, README and narrated video are accessible.

